

Отчёт по лабораторной работе № 8

Туннелирование IPv6 через IPv4

Баранов Никита Дмитриевич

2026-09-14

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретические сведения	7
4	Выполнение работы	8
4.1	На схеме два IPv6-сегмента соединены парой маршрутизаторов VyOS, между которыми используется IPv4-транзит	8
4.2	PC1 и PC2 получили IPv6-адреса 1000::a/64 и 2000::a/64	10
4.3	В нескольких консолях заданы адреса пользовательских интерфейсов и IPv4 10	12
4.4	На VyOS выполнен traceroute до IPv4-соседа и попытка обращения к удалённой IPv6-сети до завершения туннеля	12
4.5	Повторный traceroute проходит до 10	13
4.6	В двух окнах VyOS проверены таблицы маршрутизации и связность после настройки tunnel-интерфейсов	14
4.7	Финальная консольная проверка содержит ping и traceroute между 1000::/64 и 2000::/64	14
4.8	В Wireshark рядом с ICMPv6 Echo видны IPv4-пакеты с инкапсулированной нагрузкой IPv6	15
5	Выводы	16

Список иллюстраций

4.1	На схеме два IPv6-сегмента соединены парой маршрутизаторов VyOS, между которыми используется IPv4-транзит	9
4.2	PC1 и PC2 получили IPv6-адреса 1000::a/64 и 2000::a/64	11
4.3	В нескольких консолях заданы адреса пользовательских интерфейсов и IPv4 10	12
4.4	На VyOS выполнен traceroute до IPv4-соседа и попытка обращения к удалённой IPv6-сети до завершения туннеля	13
4.5	Повторный traceroute проходит до 10	13
4.6	В двух окнах VyOS проверены таблицы маршрутизации и связность после настройки tunnel-интерфейсов	14
4.7	Финальная консольная проверка содержит ping и traceroute между 1000::/64 и 2000::/64	15
4.8	В Wireshark рядом с ICMPv6 Echo видны IPv4-пакеты с инкапсулированной нагрузкой IPv6	15

Список таблиц

1 Цель работы

Настроить туннель между IPv6-сегментами через IPv4-инфраструктуру и исследовать инкапсуляцию.

2 Задание

1. Собрана топология из двух IPv6-сетей и промежуточного IPv4-канала.
2. Конечным узлам назначены IPv6-адреса и шлюзы.
3. На граничных маршрутизаторах настроены IPv4-адреса транзитной сети.
4. Создан tunnel-интерфейс с локальной и удалённой IPv4-точками.
5. В таблицы маршрутизации добавлены пути к удалённым IPv6-префиксам через туннель.
6. traceroute и ping6 показывают прохождение трафика между IPv6-сетями.
7. Wireshark подтверждает инкапсуляцию пакета IPv6 внутрь IPv4.

3 Теоретические сведения

Коммутатор изучает MAC-адреса источников и передаёт кадры через порт, за которым обнаружен адрес назначения.

4 Выполнение работы

4.1 На схеме два IPv6-сегмента соединены парой маршрутизаторов VyOS, между которыми используется IPv4-транзит

На схеме два IPv6-сегмента соединены парой маршрутизаторов VyOS, между которыми используется IPv4-транзит. Конечные VPCS подключены через отдельные коммутаторы.

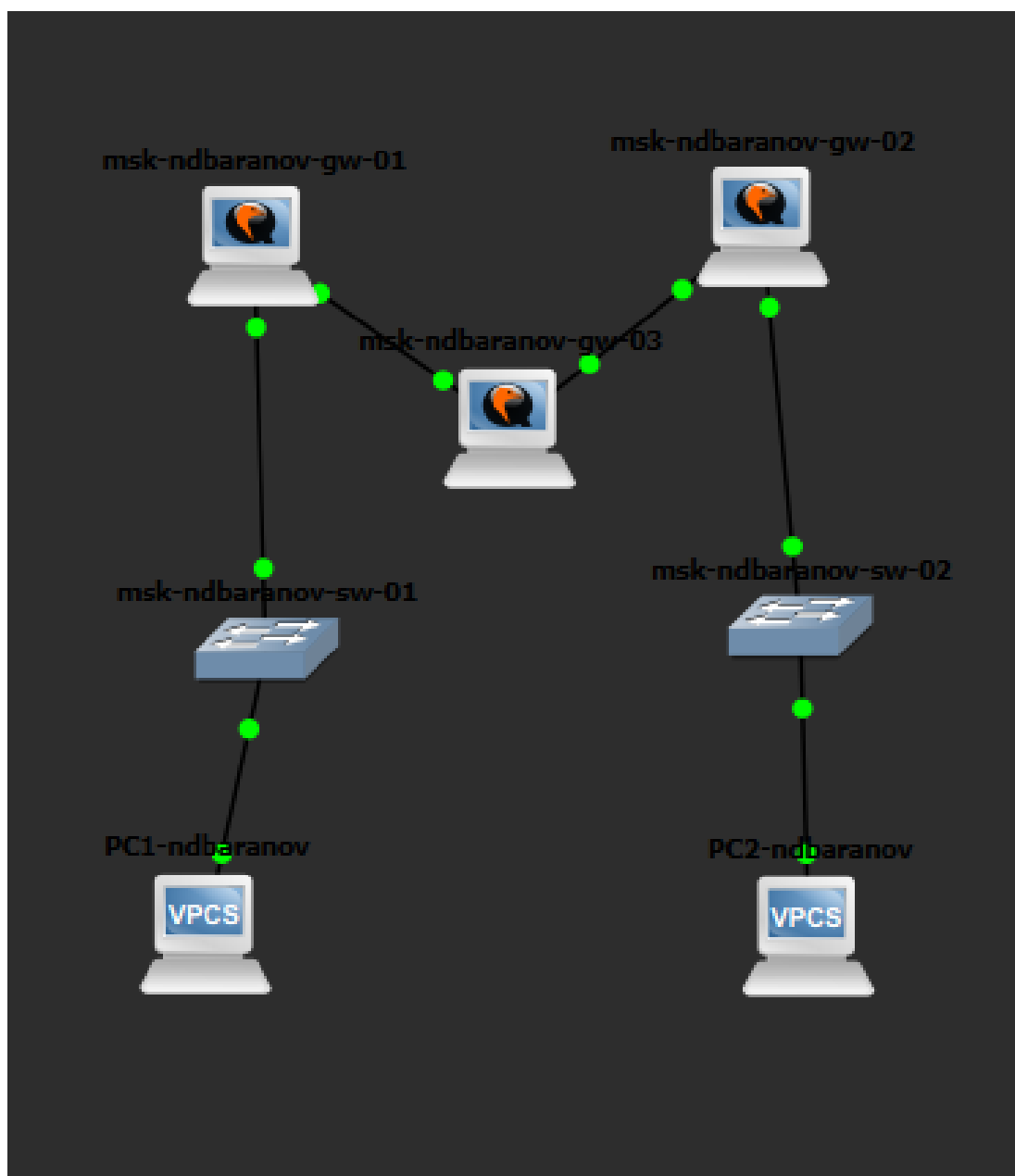


Рисунок 4.1: На схеме два IPv6-сегмента соединены парой маршрутизаторов VyOS, между которыми используется IPv4-транзит

4.2 PC1 и PC2 получили IPv6-адреса 1000::a/64 и 2000::a/64

PC1 и PC2 получили IPv6-адреса 1000::a/64 и 2000::a/64. Команда `show ipv6` также показывает `link-local` адреса, необходимые для работы соседства в локальном сегменте.

```
PC1-ndbaranov - PuTTY

Executing the startup file

Hostname is too long. (Maximum 12 characters)

VPCS> ip 1000::a/64
PC1 : 1000::a/64

VPCS> save
Saving startup configuration to startup.vpc
. done

VPCS> show ipv6

NAME           : VPCS[1]
LINK-LOCAL SCOPE : fe80::250:79ff:fe66:6800/64
GLOBAL SCOPE    : 1000::a/64
ROUTER LINK-LAYER :
MAC            : 00:50:79:66:68:00
LPORT          : 30030
RHOST:PORT      : 127.0.0.1:30031
MTU:           : 1500

VPCS>

PC2-ndbaranov - PuTTY

Executing the startup file

Hostname is too long. (Maximum 12 characters)

VPCS> ip 1002::a/64
PC1 : 1002::a/64

VPCS> save
Saving startup configuration to startup.vpc
. done

VPCS> show ipv6

NAME           : VPCS[1]
LINK-LOCAL SCOPE : fe80::250:79ff:fe66:6801/64
GLOBAL SCOPE    : 1002::a/64
ROUTER LINK-LAYER :
MAC            : 00:50:79:66:68:01
LPORT          : 30032
RHOST:PORT      : 127.0.0.1:30033
MTU:           : 1500

VPCS>
```

Рисунок 4.2: PC1 и PC2 получили IPv6-адреса 1000::a/64 и 2000::a/64

4.3 В нескольких консолях заданы адреса пользовательских интерфейсов и IPv4 10

В нескольких консолях заданы адреса пользовательских интерфейсов и IPv4 10.0.0.1/8–10.0.0.2/8 на межмаршрутизаторной линии. После commit/save базовая связность граничных узлов подготовлена.

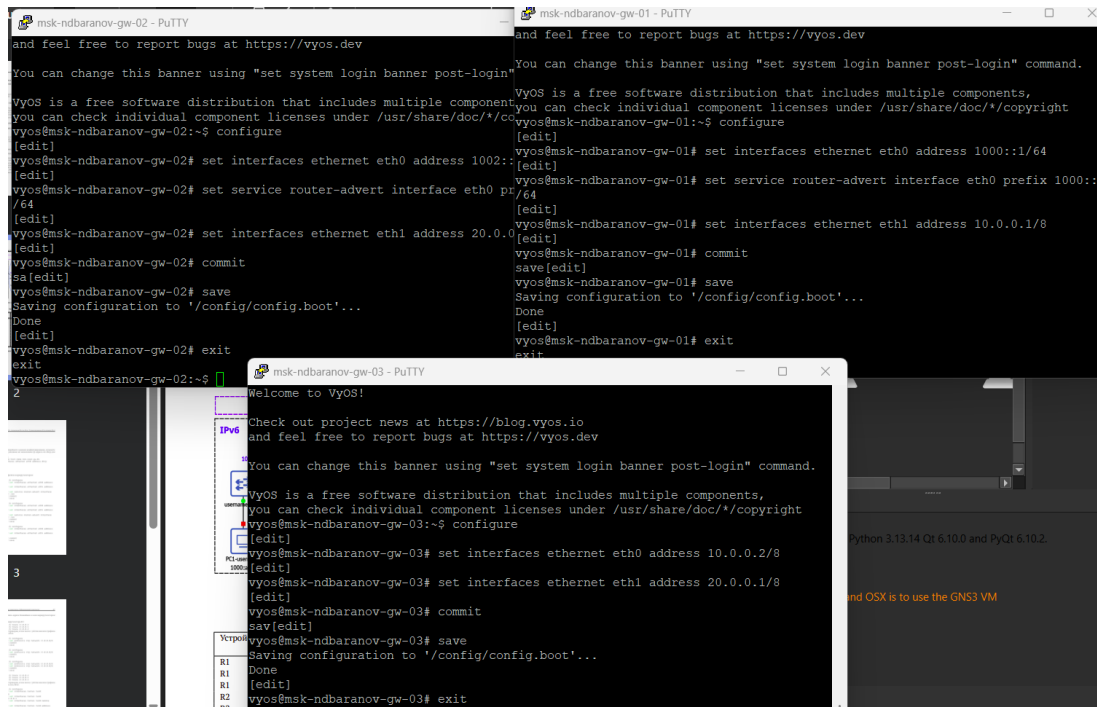
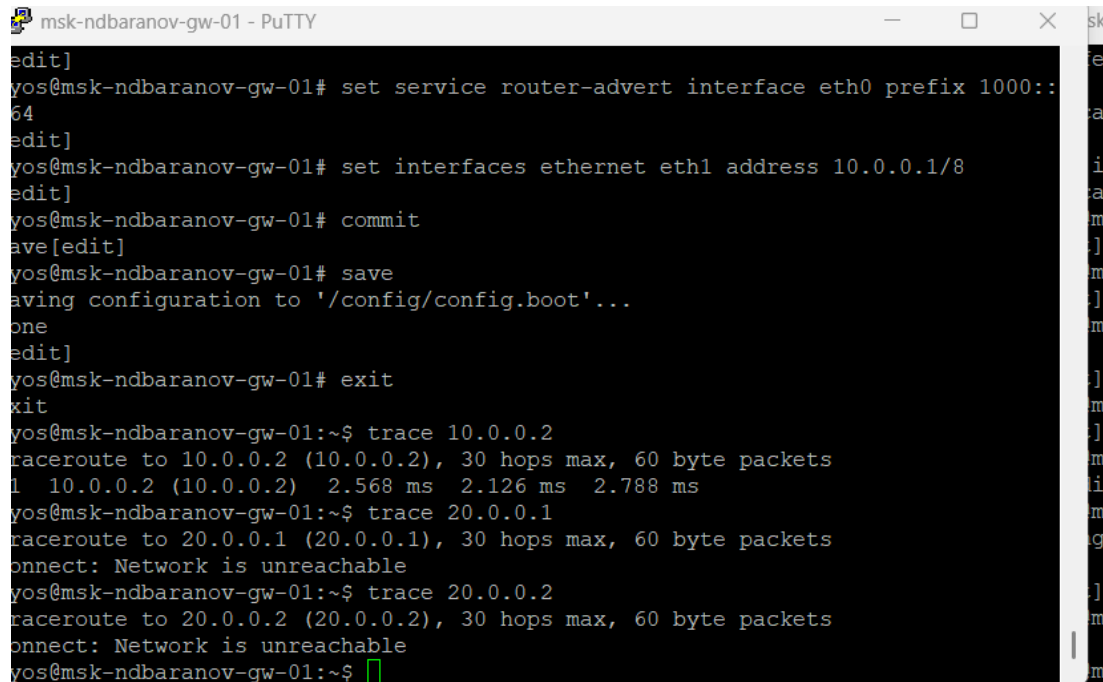


Рисунок 4.3: В нескольких консолях заданы адреса пользовательских интерфейсов и IPv4 10

4.4 На VyOS выполнен traceroute до IPv4-соседа и попытка обращения к удалённой IPv6-сети до завершения туннеля

На VyOS выполнен traceroute до IPv4-соседа и попытка обращения к удалённой IPv6-сети до завершения туннеля. Вывод отделяет доступность транспортной IPv4-

сети от маршрутизации IPv6.

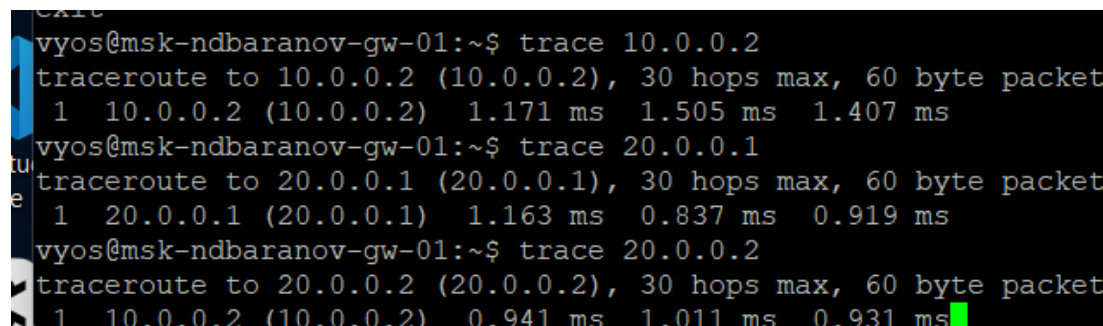


```
msk-ndbaranov-gw-01 - PuTTY
edit]
vyos@msk-ndbaranov-gw-01# set service router-advert interface eth0 prefix 1000::
64
edit]
vyos@msk-ndbaranov-gw-01# set interfaces ethernet eth1 address 10.0.0.1/8
edit]
vyos@msk-ndbaranov-gw-01# commit
save[edit]
vyos@msk-ndbaranov-gw-01# save
Saving configuration to '/config/config.boot'...
one
edit]
vyos@msk-ndbaranov-gw-01# exit
xit
vyos@msk-ndbaranov-gw-01:~$ trace 10.0.0.2
Traceroute to 10.0.0.2 (10.0.0.2), 30 hops max, 60 byte packets
 0  10.0.0.2 (10.0.0.2)  2.568 ms  2.126 ms  2.788 ms
vyos@msk-ndbaranov-gw-01:~$ trace 20.0.0.1
Traceroute to 20.0.0.1 (20.0.0.1), 30 hops max, 60 byte packets
 0  connect: Network is unreachable
vyos@msk-ndbaranov-gw-01:~$ trace 20.0.0.2
Traceroute to 20.0.0.2 (20.0.0.2), 30 hops max, 60 byte packets
 0  connect: Network is unreachable
vyos@msk-ndbaranov-gw-01:~$
```

Рисунок 4.4: На VyOS выполнен traceroute до IPv4-соседа и попытка обращения к удалённой IPv6-сети до завершения туннеля

4.5 Повторный traceroute проходит до 10

Повторный traceroute проходит до 10.0.0.2 и показывает один промежуточный переход. Таким образом, адреса локальной и удалённой точек будущего туннеля взаимно достижимы.

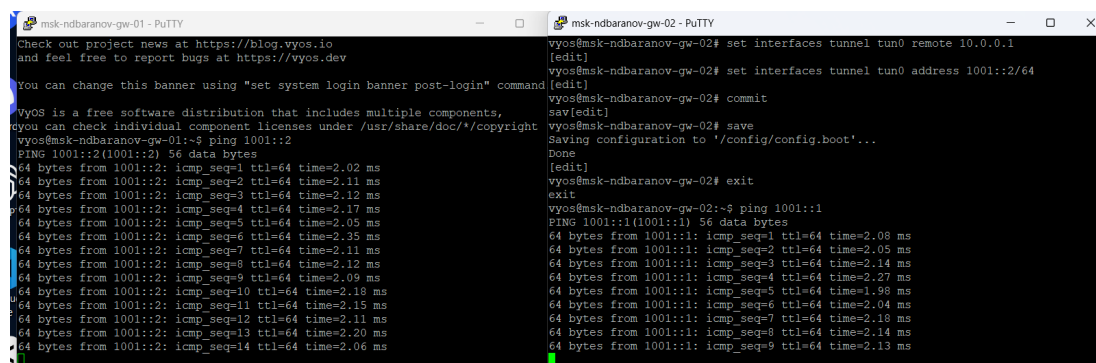


```
vyos@msk-ndbaranov-gw-01:~$ trace 10.0.0.2
Traceroute to 10.0.0.2 (10.0.0.2), 30 hops max, 60 byte packet
 0  1  10.0.0.2 (10.0.0.2)  1.171 ms  1.505 ms  1.407 ms
vyos@msk-ndbaranov-gw-01:~$ trace 20.0.0.1
Traceroute to 20.0.0.1 (20.0.0.1), 30 hops max, 60 byte packet
 0  1  20.0.0.1 (20.0.0.1)  1.163 ms  0.837 ms  0.919 ms
vyos@msk-ndbaranov-gw-01:~$ trace 20.0.0.2
Traceroute to 20.0.0.2 (20.0.0.2), 30 hops max, 60 byte packet
 0  1  10.0.0.2 (10.0.0.2)  0.941 ms  1.011 ms  0.931 ms
```

Рисунок 4.5: Повторный traceroute проходит до 10

4.6 В двух окнах VyOS проверены таблицы маршрутизации и связность после настройки tunnel-интерфейсов

В двух окнах VyOS проверены таблицы маршрутизации и связность после настройки tunnel-интерфейсов. Появившиеся пути направляют удалённые IPv6-префиксы через логический канал.



The image shows two terminal windows side-by-side. The left window, titled 'msk-ndbaranov-gw-01 - PuTTY', shows the VyOS login banner and a successful ping test from 1001::2 to 1001::1. The right window, titled 'msk-ndbaranov-gw-02 - PuTTY', shows the configuration of a tunnel interface 'tun0' with remote address 10.0.0.1 and local address 1001::2/64, followed by a successful ping test from 1001::1 to 1001::2.

```
msk-ndbaranov-gw-01 - PuTTY
Check out project news at https://blog.vyos.io
and feel free to report bugs at https://vyos.dev

You can change this banner using "set system login banner post-login" command.

VyOS is a free software distribution that includes multiple components,
you can check individual component licenses under /usr/share/doc/*/copyright
vyos@msk-ndbaranov-gw-01:~$ ping 1001::1
PING 1001::1(1001::1) 56 data bytes:
64 bytes from 1001::1: icmp_seq=1 ttl=64 time=2.02 ms
64 bytes from 1001::1: icmp_seq=2 ttl=64 time=2.11 ms
64 bytes from 1001::1: icmp_seq=3 ttl=64 time=2.12 ms
64 bytes from 1001::1: icmp_seq=4 ttl=64 time=2.17 ms
64 bytes from 1001::1: icmp_seq=5 ttl=64 time=2.05 ms
64 bytes from 1001::1: icmp_seq=6 ttl=64 time=2.35 ms
64 bytes from 1001::1: icmp_seq=7 ttl=64 time=2.11 ms
64 bytes from 1001::1: icmp_seq=8 ttl=64 time=2.12 ms
64 bytes from 1001::1: icmp_seq=9 ttl=64 time=2.09 ms
64 bytes from 1001::1: icmp_seq=10 ttl=64 time=2.18 ms
64 bytes from 1001::1: icmp_seq=11 ttl=64 time=2.15 ms
64 bytes from 1001::1: icmp_seq=12 ttl=64 time=2.11 ms
64 bytes from 1001::1: icmp_seq=13 ttl=64 time=2.20 ms
64 bytes from 1001::1: icmp_seq=14 ttl=64 time=2.06 ms

msk-ndbaranov-gw-02 - PuTTY
vyos@msk-ndbaranov-gw-02# set interfaces tunnel tun0 remote 10.0.0.1
[edit]
vyos@msk-ndbaranov-gw-02# set interfaces tunnel tun0 address 1001::2/64
[edit]
vyos@msk-ndbaranov-gw-02# commit
save[edit]
vyos@msk-ndbaranov-gw-02# save
Saving configuration to '/config/config.boot'...
Done
[edit]
vyos@msk-ndbaranov-gw-02# exit
exit
vyos@msk-ndbaranov-gw-02:~$ ping 1001::2
PING 1001::2(1001::2) 56 data bytes:
64 bytes from 1001::2: icmp_seq=1 ttl=64 time=2.08 ms
64 bytes from 1001::2: icmp_seq=2 ttl=64 time=2.05 ms
64 bytes from 1001::2: icmp_seq=3 ttl=64 time=2.14 ms
64 bytes from 1001::2: icmp_seq=4 ttl=64 time=2.27 ms
64 bytes from 1001::2: icmp_seq=5 ttl=64 time=1.98 ms
64 bytes from 1001::2: icmp_seq=6 ttl=64 time=2.04 ms
64 bytes from 1001::2: icmp_seq=7 ttl=64 time=2.18 ms
64 bytes from 1001::2: icmp_seq=8 ttl=64 time=2.14 ms
64 bytes from 1001::2: icmp_seq=9 ttl=64 time=2.13 ms
```

Рисунок 4.6: В двух окнах VyOS проверены таблицы маршрутизации и связность после настройки tunnel-интерфейсов

4.7 Финальная консольная проверка содержит ping и traceroute между 1000::/64 и 2000::/64

Финальная консольная проверка содержит ping и traceroute между 1000::/64 и 2000::/64. Маршрут проходит через граничные маршрутизаторы, хотя промежуточная сеть использует IPv4.

```

[edit]
vyos@msk-ndbaranov-gw-01# commit
sa[edit]
vyos@msk-ndbaranov-gw-01# save
Saving configuration to '/config/config.boot'...
Done
[edit]
vyos@msk-ndbaranov-gw-01# exit
exit
vyos@msk-ndbaranov-gw-01~$ show ipv6 route
Codes: K - kernel route, C - connected, S - static, R - RIPng,
       O - OSPFv3, I - IS-IS, B - BGP, N - NHRP, T - Table,
       v - VNC, V - VNC-Direct, A - Babel, D - SHARP, F - FBR,
       f - OpenFabric,
       > - selected route, * - FIB route, q - queued, r - rejected, b - backup
C>* 1000::/64 is directly connected, eth0, 00:07:07
C>* 1001::/64 is directly connected, tun0, 00:01:55
S>* 1002::/64 [1/0] via 1001::2, tun0, weight 1, 00:00:22
C>* fe80::/64 is directly connected, tun0, 00:01:55
C>* fe80::/64 is directly connected, eth0, 00:10:55
C>* fe80::/64 is directly connected, eth1, 00:10:56
C>* fe80::/64 is directly connected, lo, 00:10:58
vyos@msk-ndbaranov-gw-01~$

[edit]
vyos@msk-ndbaranov-gw-02# commit
sav[edit]
vyos@msk-ndbaranov-gw-02# save
Saving configuration to '/config/config.boot'...
Done
[edit]
vyos@msk-ndbaranov-gw-02# exit
exit
vyos@msk-ndbaranov-gw-02~$ show ipv6 route
Codes: K - kernel route, C - connected, S - static, R - RIPng,
       O - OSPFv3, I - IS-IS, B - BGP, N - NHRP, T - Table,
       v - VNC, V - VNC-Direct, A - Babel, D - SHARP, F - FBR,
       f - OpenFabric,
       > - selected route, * - FIB route, q - queued, r - rejected, b - backup
S>* 1000::/64 [1/0] via 1001::1, tun0, weight 1, 00:00:20
C>* 1001::/64 is directly connected, tun0, 00:01:50
C>* 1002::/64 is directly connected, eth0, 00:06:47
C>* fe80::/64 is directly connected, tun0, 00:01:50
C>* fe80::/64 is directly connected, eth0, 00:08:05
C>* fe80::/64 is directly connected, eth1, 00:08:06
C>* fe80::/64 is directly connected, lo, 00:08:07
vyos@msk-ndbaranov-gw-02~$

```

Рисунок 4.7: Финальная консольная проверка содержит ping и traceroute между 1000::/64 и 2000::/64

4.8 В Wireshark рядом с ICMPv6 Echo видны IPv4-пакеты с инкапсулированной нагрузкой IPv6

В Wireshark рядом с ICMPv6 Echo видны IPv4-пакеты с инкапсулированной нагрузкой IPv6. Совпадение обменов по времени подтверждает перенос исходного IPv6-пакета внутри IPv4.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.2	224.0.0.9	RIPv2	66	Response
2	23.003939	10.0.0.2	224.0.0.9	RIPv2	66	Response
3	42.408671	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request id=0x0c00, seq=1/256, ttl=64 (reply in 4)
4	42.409395	10.0.0.2	10.0.0.1	ICMP	98	Echo (ping) reply id=0x0c00, seq=1/256, ttl=64 (request in 3)
5	43.408183	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request id=0x0c00, seq=2/512, ttl=64 (reply in 6)
6	43.408890	10.0.0.2	10.0.0.1	ICMP	98	Echo (ping) reply id=0x0c00, seq=2/512, ttl=64 (request in 5)
7	44.410260	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request id=0x0c00, seq=3/768, ttl=64 (reply in 8)
8	44.410943	10.0.0.2	10.0.0.1	ICMP	98	Echo (ping) reply id=0x0c00, seq=3/768, ttl=64 (request in 7)
9	45.410328	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request id=0x0c00, seq=4/1024, ttl=64 (reply in 10)
10	45.411040	10.0.0.2	10.0.0.1	ICMP	98	Echo (ping) reply id=0x0c00, seq=4/1024, ttl=64 (request in 9)
11	46.006056	10.0.0.2	224.0.0.9	RIPv2	66	Response
12	46.411895	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request id=0x0c00, seq=5/1280, ttl=64 (reply in 13)
13	46.412515	10.0.0.2	10.0.0.1	ICMP	98	Echo (ping) reply id=0x0c00, seq=5/1280, ttl=64 (request in 12)
14	47.413772	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request id=0x0c00, seq=6/1536, ttl=64 (reply in 15)
15	47.414600	10.0.0.2	10.0.0.1	ICMP	98	Echo (ping) reply id=0x0c00, seq=6/1536, ttl=64 (request in 14)
16	47.736717	0c:20:e8:17:00:01	0c:20:01:3a:00:00	ARP	60	Who has 10.0.0.2? Tell 10.0.0.1
17	47.737536	0c:20:01:3a:00:00	0c:20:e8:17:00:01	ARP	60	10.0.0.2 is at 0c:20:01:3a:00:00
18	47.743408	0c:20:01:3a:00:00	0c:20:e8:17:00:01	ARP	60	Who has 10.0.0.1? Tell 10.0.0.2

Рисунок 4.8: В Wireshark рядом с ICMPv6 Echo видны IPv4-пакеты с инкапсулированной нагрузкой IPv6

5 Выводы

Цель работы достигнута. Настроить туннель между IPv6-сегментами через IPv4-инфраструктуру и исследовать инкапсуляцию. Полученные результаты подтверждают работоспособность собранной схемы и корректность выполненной настройки.